

CSTDIO_REFERENCE

Core Definition

`<cstdio>` is the C++ wrapper for C's `<stdio.h>`: `FILE*`-based I/O — `printf` / `scanf` formatting, `fopen` / `fread` / `fwrite`, and the standard streams `stdin` / `stdout` / `stderr`. It coexists with C++ streams (they share buffers by default) and remains relevant for C interop, `snprintf`-style formatting into fixed buffers, and low-level file work.

#CPP #CSTDIO #PRINTF #SCANF #FILE #C-IO #FOPEN #LEGACY

COMPLETE CSTDIO QUICK REFERENCE

```
// =====
// FILE ACCESS – Target | Operation | Output
// =====
std::fopen(name, mode)           // path+mode | Open a file           | Returns FILE* or nullptr
std::freopen(name, mode, stream) // + stream | Reopen a stream        | Returns FILE* or nullptr
std::fclose(f)                  // FILE*    | Close and flush        | Returns 0 or EOF
std::fflush(f)                  // FILE*    | Flush output buffer     | Returns 0 or EOF; nullptr = flush all
std::setbuf(f, buf)             // FILE*+buf | Set buffer              | Returns void; buf size must be BUFSIZ
std::setvbuf(f, buf, mode, size) // + mode   | Set buffering mode      | Returns 0 on success
// mode: _IOFBF full, _IOLBF line, _IONBF none

// =====
// FILE MODES (fopen)
// =====
"r" // Read; file must exist
"w" // Write; create or TRUNCATE
"a" // Append; create if missing; writes always go to the end
"r+" // Read/write; file must exist
"w+" // Read/write; create or truncate
"a+" // Read/write; append-only writes
"b" // Append to any of the above for binary: "rb", "w+b", "ab"
"x" // C11: append to "w"/"w+" for exclusive create – fail if the file exists

// =====
// FORMATTED OUTPUT
// =====
std::printf(fmt, ...)           // format | Write to stdout       | Returns chars written or negative
std::fprintf(f, fmt, ...)       // + FILE* | Write to a stream     | Returns chars written
std::sprintf(buf, fmt, ...)      // + buffer | Write to a buffer     | UNSAFE – no size limit
std::snprintf(buf, n, fmt, ...)  // + size  | Bounded write         | Returns chars that WOULD be written
std::vprintf / vfprintf / vsprintf / vsnprintf // va_list variants (<cstdarg>)

// =====
// FORMATTED INPUT
// =====
std::scanf(fmt, ...)            // format | Read from stdin       | Returns items assigned, or EOF
std::fscanf(f, fmt, ...)        // + FILE* | Read from a stream    | Returns items assigned
std::sscanf(str, fmt, ...)       // + string | Read from a string    | Returns items assigned
std::vscanf / vfscanf / vsscanf // va_list variants

// =====
// CHARACTER I/O
// =====
std::fgetc(f)                   // FILE*    | Read one char         | Returns int or EOF
std::getc(f)                    // FILE*    | Read one char         | May be a macro; same semantics
std::getchar()                  // none     | Read from stdin       | Returns int or EOF
std::fputc(c, f)                // char+FILE* | Write one char        | Returns the char or EOF
std::putc(c, f)                 // char+FILE* | Write one char        | May be a macro
std::putchar(c)                 // char     | Write to stdout        | Returns the char or EOF
```

```

std::ungetc(c, f) // char+FILE* | Push one char back | Returns the char or EOF

// =====
// STRING I/O
// =====
std::fgets(buf, n, f) // buf+n | Read a line | Returns buf or nullptr; KEEPS '\n'
std::fputs(s, f) // str+FILE* | Write a string | Returns non-negative or EOF; adds no
'\n'
std::puts(s) // str | Write + newline | Returns non-negative or EOF
// std::gets - REMOVED in C++14. Never usable; unbounded overflow by design.

// =====
// BINARY (DIRECT) I/O
// =====
std::fread(ptr, size, count, f) // buf+dims | Read count items | Returns items READ (may be < count)
std::fwrite(ptr, size, count, f) // buf+dims | Write count items | Returns items WRITTEN

// =====
// FILE POSITIONING
// =====
std::ftell(f) // FILE* | Current position | Returns long or -1L
std::fseek(f, off, origin) // offset | Seek | Returns 0 on success
// origin: SEEK_SET, SEEK_CUR, SEEK_END
std::fgetpos(f, &pos) // fpos_t* | Save position | Returns 0; handles large files
std::fsetpos(f, &pos) // fpos_t* | Restore position | Returns 0
std::rewind(f) // FILE* | Seek to start | Returns void; also clears error flags

// =====
// ERROR HANDLING
// =====
std::feof(f) // FILE* | End-of-file flag | Returns non-zero if set
std::ferror(f) // FILE* | Error flag | Returns non-zero if set
std::clearerr(f) // FILE* | Clear both flags | Returns void
std::perror(msg) // string | Print msg + errno | Returns void; writes to stderr

// =====
// FILE OPERATIONS
// =====
std::remove(name) // path | Delete a file | Returns 0 on success
std::rename(old, new) // two paths | Rename / move | Returns 0 on success
std::tmpfile() // none | Open a temp file | Returns FILE*; auto-deleted on close
std::tmpnam(buf) // buf | Generate a temp name | UNSAFE (race); prefer tmpfile

// =====
// STANDARD STREAMS & CONSTANTS
// =====
stdin / stdout / stderr // FILE* - the standard streams
EOF // End-of-file indicator (typically -1)
BUFSIZ // Default buffer size
FILENAME_MAX / FOPEN_MAX // Path length / simultaneous open limits
L_tmpnam / TMP_MAX // Temp name buffer size / unique names available
SEEK_SET / SEEK_CUR / SEEK_END // fseek origins
_IOFBF / _IOLBF / _IONBF // setvbuf modes
NULL // Null pointer macro (prefer nullptr)
std::FILE / std::fpos_t / std::size_t

```

PRINTF FORMAT SPECIFIERS

```
%[flags][width][.precision][length]conversion
```

CONVERSIONS

%d %i	signed decimal int	%u	unsigned decimal
%o	unsigned octal	%x %X	unsigned hex (lower/upper)
%f %F	decimal float (3.140000)	%e %E	scientific (3.14e+00)
%g %G	shorter of %f/%e	%a %A	hex float
%c	character	%s	C-string
%p	pointer	%%	literal %

```

%n      store chars written so far into an int* (dangerous; often disabled)

FLAGS
- left-justify      + always show sign
(space) space for positive  # alternate form (0x for %x, force '.' for floats)
0 zero-pad the field

WIDTH / PRECISION
8      minimum field width      .3      precision
*      take the value from the argument list: printf("%.f", 10, 2, x)

LENGTH MODIFIERS
hh char      h short      l long      ll long long
j intmax_t   z size_t     t ptrdiff_t L long double

COMMON COMBINATIONS
%5d      width 5, right-aligned      %-10s width 10, left-aligned
%05.2f   zero-padded, 2 decimals     %+d always signed
%#x      0x-prefixed hex             %zu size_t
%lld     long long                   %.*s bounded string

```

SCANF FORMAT SPECIFIERS

```

%d %i %u %o %x  integers (%i auto-detects base from 0x/0 prefix)
%f %e %g        float      %lf %le %lg double      %Lf long double
%c              one char (NO whitespace skip, no null terminator added)
%s              whitespace-delimited word – ALWAYS give a width: %31s
%[abc]          scanfset: match only these characters
%[^\n]          scanfset: match everything except newline
%n              chars consumed so far
%*d             assign-suppress: parse and discard
literal space   matches any amount of whitespace, including none

```

COMMON PATTERNS & EXAMPLES

Reading a File Line by Line

```

#include <cstdio>

std::FILE* f = std::fopen("data.txt", "r");
if (!f) { std::perror("fopen"); return 1; }

char line[256];
while (std::fgets(line, sizeof(line), f)) {
    // fgets KEEPS the newline; strip it if you don't want it
    line[std::strcspn(line, "\n")] = '\0';
    std::printf("%s\n", line);
}

if (std::ferror(f)) std::fprintf(stderr, "read error\n");
std::fclose(f);

```

Safe Formatting into a Buffer

```
#include <cstdio>

char buf[64];

// UNSAFE – no bound:
// std::sprintf(buf, "%s=%d", key, value);

// Bounded, and the return value tells you if it fit:
int n = std::snprintf(buf, sizeof(buf), "%s=%d", key, value);
if (n < 0) {
    // encoding error
} else if (static_cast<std::size_t>(n) >= sizeof(buf)) {
    // TRUNCATED – n is the length it WOULD have needed
}
```

snprintf returns the length the full output would have had, not the number written. That is what makes the two-pass sizing idiom below work.

Two-Pass Sizing

```
#include <cstdio>
#include <string>
#include <vector>

std::string formatToString(const char* fmt, int a, double b) {
    int n = std::snprintf(nullptr, 0, fmt, a, b); // Measure
    if (n < 0) return {};
    std::string out(static_cast<std::size_t>(n), '\0');
    std::snprintf(out.data(), static_cast<std::size_t>(n) + 1, fmt, a, b);
    return out;
}
// C++20: std::format("{} {}", a, b) does this safely and type-checked.
```

Binary Read/Write

```
#include <cstdio>
#include <vector>

struct Record { int id; double value; };

bool writeAll(const char* path, const std::vector<Record>& recs) {
    std::FILE* f = std::fopen(path, "wb"); // 'b' matters
    if (!f) return false;
    std::size_t n = std::fwrite(recs.data(), sizeof(Record), recs.size(), f);
    std::fclose(f);
    return n == recs.size(); // fwrite may write fewer
}

std::vector<Record> readAll(const char* path) {
    std::FILE* f = std::fopen(path, "rb");
    if (!f) return {};
    std::fseek(f, 0, SEEK_END);
    long bytes = std::ftell(f);
    std::rewind(f);

    std::vector<Record> recs(static_cast<std::size_t>(bytes) / sizeof(Record));
    std::size_t got = std::fread(recs.data(), sizeof(Record), recs.size(), f);
    recs.resize(got);
    std::fclose(f);
    return recs;
}
```

Parsing with `sscanf`

```
#include <cstdio>

const char* line = "widget 42 3.14";
char name[32];
int qty;
double price;

// The width on %s is MANDATORY – without it, a long token overflows `name`
int matched = std::sscanf(line, "%31s %d %lf", name, &qty, &price);
if (matched != 3) { /* parse failed at field `matched` */ }
```

`scanf` returns the number of successful assignments, not the number of fields in the format. Always check it against the expected count.

The `scanf` Newline Trap

```
int n;
char line[128];

std::scanf("%d", &n);           // Leaves the '\n' in the buffer
// std::fgets(line, sizeof(line), stdin); // Would read an empty line

// Consume the rest of the line first:
int c; while ((c = std::getchar()) != '\n' && c != EOF) {}
std::fgets(line, sizeof(line), stdin);

// Or read whole lines and sscanf them – usually the better design.
```

Reading a Line Safely, Whole

```
// %[^\n] with a width reads up to (but not including) the newline:
char buf[256];
if (std::scanf("%255[^\n]", buf) == 1) {
    std::getchar();           // Consume the newline
}
// fgets is simpler and does not have scanf's edge cases.
```

RAII Wrapper for `FILE*`

```
#include <cstdio>
#include <memory>

struct FileCloser {
    void operator()(std::FILE* f) const noexcept { if (f) std::fclose(f); }
};
using FilePtr = std::unique_ptr<std::FILE, FileCloser>;

FilePtr openFile(const char* path, const char* mode) {
    return FilePtr(std::fopen(path, mode));
}

// FilePtr f = openFile("x.txt", "r");
// if (f) std::fgets(buf, sizeof(buf), f.get());
// Closed automatically, including on an exception.
```

Error Reporting

```
#include <cstdio>
#include <cerrno>
#include <cstring>

std::FILE* f = std::fopen("missing.txt", "r");
if (!f) {
    std::perror("open failed");           // "open failed: No such file..."
    // or:
    std::fprintf(stderr, "open failed: %s\n", std::strerror(errno));
}
```

Buffering Control

```
std::setvbuf(stdout, nullptr, _IONBF, 0);    // Unbuffered – every write hits the device
std::setvbuf(stdout, nullptr, _IOLBF, BUFSIZ); // Line buffered
char buf[8192];
std::setvbuf(stdout, buf, _IOFBF, sizeof(buf)); // Fully buffered with your own memory
// Call setvbuf BEFORE any I/O on the stream.
```

Mixing with C++ Streams

```
#include <cstdio>
#include <iostream>

// SAFE by default: sync_with_stdio(true) keeps printf and cout in order
std::printf("first\n");
std::cout << "second\n";

// After std::ios_base::sync_with_stdio(false), the two buffer independently
// and output can interleave incorrectly. Pick one API per program.
```

IMPORTANT CONCEPTS

printf Is Not Type-Safe

The format string is parsed at runtime; the arguments are passed through varargs with no checking. A `%d` with a long, `long`, or a `%s` with a non-string, is undefined behavior — often a crash, sometimes a security hole. GCC and Clang catch many cases with `-Wformat` `-Wformat-security`, but only for literal format strings.

Never Pass User Input as a Format String

```
std::printf(userInput);           // FORMAT STRING VULNERABILITY
std::printf("%s", userInput);     // Correct
```

A `%n` in attacker-controlled input can write to memory. This is a classic exploit class.

sprintf Has No Safe Use

There is no way to bound it. `snprintf` is the same function with a size parameter; use it always. `gets` was worse still and was removed from the language in C++14.

`fgets` Keeps the Newline

Unlike `std::getline`, `fgets` stores the `'\n'` if it fit. Strip it explicitly. If the line was longer than the buffer, there is no newline and the rest of the line is still waiting — check for it.

`scanf("%s")` Without a Width Is `gets`

`%s` reads until whitespace with no bound. Always write `%31s` for a `char[32]`. The width excludes the null terminator that `scanf` adds.

`feof()` Is Not a Loop Condition

```
while (!std::feof(f)) { std::fgets(line, n, f); process(line); } // Processes the last line twice
while (std::fgets(line, n, f)) { process(line); }                // Correct
```

`feof` becomes true only *after* a read has already failed — the same trap as `std::cin.eof()`.

`fread/fwrite` Return Item Counts

They return the number of complete *items*, not bytes. A short return means end of file or an error — distinguish with `feof` / `ferror`.

`ftell` Returns `long`

On 32-bit builds that caps at 2 GB. Use `fgetpos` / `fsetpos` (which use the opaque `fpos_t`) for large files, or the platform's 64-bit variants.

Text Mode Translates on Windows

Without `"b"`, `\n` becomes `\r\n` on write and back on read, and `ftell` values in text mode are not byte offsets. Any non-text file needs `"rb"` / `"wb"`.

`%lf` in `printf` vs `scanf`

In `printf`, `float` is promoted to `double`, so `%f` handles both and `%lf` is merely tolerated. In `scanf` the distinction is real: `%f` writes to a `float*`, `%lf` to a `double*`. Getting it wrong corrupts memory.

The C++ Alternatives Are Better

`std::format` (C++20) and `std::print` (C++23) give the same concise, positional style with compile-time checking, no varargs, and no buffer to size. `std::ostringstream` and `std::to_chars` cover the rest. Reach for `<cstdio>` when interfacing with C code, when you need `FILE*` specifically, or when a platform API hands you one.

PRINTF ↔ MODERN C++ MAPPING

printf	std::format (C++20)
"%d"	"{}"
"%5d"	"{:5}"
"%-10s"	"{:<10}"
"%05.2f"	"{:05.2f}"
"%.3f"	"{:.3f}"
"%#x"	"{:#x}"
"%e"	"{:e}"
"%+d"	"{:+}"
"%c"	"{}"
"%%"	"{{}}"
snprintf(buf, n, ...)	std::format_to_n(buf, n, ...)
printf(...)	std::print(...) (C++23)

BEST PRACTICES

1. Use `snprintf`, never `sprintf`; check the return against the buffer size
2. Always bound `%s` in `scanf` — `%31s` for a `char[32]`
3. Never pass a non-literal as the format string
4. Loop on the read, not on `feof()`
5. Wrap `FILE*` in a `unique_ptr` with a custom deleter so it closes on every path
6. Open non-text files with `"b"`
7. Check `fread` / `fwrite` return counts — short transfers are normal
8. Enable `-Wformat` `-Wformat=2` and treat format warnings as errors
9. Use `fgetpos` / `fsetpos` for files that may exceed 2 GB
10. Call `setvbuf` before any I/O on the stream
11. Don't mix `printf` and `cout` after `sync_with_stdio(false)`
12. Prefer `std::format` / `std::print` in new C++ code; keep `<cstdio>` for C boundaries

RELATED HEADERS

```
#include <cstdio>      // FILE*, printf, fopen, fread ...
#include <cstdarg>      // va_list, for vprintf-style wrappers
#include <cerrno>       // errno
#include <cstring>      // strerror, strchr
#include <stdint.h>     // PRId64 etc. — portable printf specifiers for fixed-width types
#include <wchar.h>      // wprintf, fgetws — wide-character stdio
#include <format>       // C++20: std::format — type-safe replacement
#include <print>        // C++23: std::print / println
#include <fstream>      // C++ file streams
#include <filesystem>   // C++17: remove, rename, file_size, exists
```

EXTERNAL RESOURCES

- **C I/O library:** <https://en.cppreference.com/w/cpp/io/c>
- **printf format:** <https://en.cppreference.com/w/cpp/io/c/fprintf>
- **scanf format:** <https://en.cppreference.com/w/cpp/io/c/fscanf>

Standard: C++11+ (C99/C11 library base); `gets` removed in C++14 **Last Updated:** September 2026